==========

CAPSTONE PROJECT

NOTEBOOK 40: FULL-161 RARE-TAIL FALLBACK ROUTER

==========

Purpose:

This notebook builds the Stage-2 fallback mechanism needed to

move from the completed candidate-150 system toward a full

161-label pipeline.

The goal is to:

1. Load the rare-tail audit outputs

2. Map each rare-tail label to the nearest candidate ancestor

3. Compute training co-occurrence statistics for fallback routing

4. Build anchor-to-rare-tail routing tables

5. Estimate routing coverage using ground-truth hierarchy

6. Save the Stage-2 fallback artifacts for the final full-genre pipeline

==========

In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import math
import numpy as np
import pandas as pd

SEED = 42
np.random.seed(SEED)

print("Seed set to:", SEED)
```

```
Seed set to: 42
```

In [2]:

```
# ============================================================
# 2. LOAD CORE TABLES
# ============================================================

genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
strategy_df = pd.read_csv("../data/processed/full161_all_genre_strategy_table.csv")
rare_tail_summary_df =
pd.read_csv("../data/processed/full161_rare_tail_summary.csv")
rare_tail_pairs_df =
pd.read_csv("../data/processed/full161_rare_tail_track_label_pairs.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

print("Genre inventory shape:", genre_inventory_df.shape)
print("Strategy table shape:", strategy_df.shape)
print("Rare-tail summary shape:", rare_tail_summary_df.shape)
print("Rare-tail track-label pairs shape:", rare_tail_pairs_df.shape)
print("Full master table shape:", full_master_df.shape)
```

```
display(genre_inventory_df.head())
display(strategy_df.head())
display(rare_tail_summary_df.head())
display(rare_tail_pairs_df.head())
display(full_master_df.head())
```

```
Genre inventory shape: (163, 11)
Strategy table shape: (163, 18)
Rare-tail summary shape: (13, 11)
Rare-tail track-label pairs shape: (124, 13)
Full master table shape: (81574, 170)
```

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_
0	38	Experimental	NaN	NaN	38	Experimental	
1	15	Electronic	NaN	NaN	15	Electronic	
2	12	Rock	NaN	NaN	12	Rock	
3	1235	Instrumental	NaN	NaN	1235	Instrumental	1
4	10	Pop	NaN	NaN	10	Pop	

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_
0	38	Experimental	NaN	NaN	38	Experimental	
1	15	Electronic	NaN	NaN	15	Electronic	
2	12	Rock	NaN	NaN	12	Rock	
3	1235	Instrumental	NaN	NaN	1235	Instrumental	1
4	10	Pop	NaN	NaN	10	Pop	

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	total_coun
0	176	Pacific	2.0	International	2	International	2
1	1060	Tango	46.0	Latin America	2	International	2
2	465	Musical Theater	20.0	Spoken	20	Spoken	1
3	189	Talk Radio	65.0	Radio	20	Spoken	1
4	1032	Turkish	102.0	Middle East	2	International	1

	track_id	split	subset	genre_top	title	audio_path
0	13077	validation	large	NaN	Thanam-Kalyani	../data/raw/audio/fma_large\013\013077.mp3
1	16930	test	large	NaN	Open Session	../data/raw/audio/fma_large\016\016930.mp3
2	19777	validation	large	NaN	Custer: "If I Were An Indian..."	../data/raw/audio/fma_large\019\019777.mp3
3	19778	validation	large	NaN	Custer's Ghost To Sitting Bull	../data/raw/audio/fma_large\019\019778.mp3
4	19779	validation	large	NaN	Sitting Bull: "Do You Know Who I Am?"	../data/raw/audio/fma_large\019\019779.mp3

	track_id	split	subset	genre_top	title	audio_path	a
0	20	training	large	NaN	Spiritual Level	../data/raw/audio/fma_large\000\000020.mp3	
1	26	training	large	NaN	Where is your Love?	../data/raw/audio/fma_large\000\000026.mp3	
2	30	training	large	NaN	Too Happy	../data/raw/audio/fma_large\000\000030.mp3	
3	46	training	large	NaN	Yosemite	../data/raw/audio/fma_large\000\000046.mp3	
4	48	training	large	NaN	Light of Light	../data/raw/audio/fma_large\000\000048.mp3	

5 rows × 170 columns

In [3]:

```
# ============================================================
# 3. PREPARE LOOKUP STRUCTURES
# ============================================================

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)

for col in ["parent_id", "root_genre_id", "top_level_flag"]:
    if col in genre_inventory_df.columns:
        genre_inventory_df[col] = pd.to_numeric(genre_inventory_df[col],
errors="coerce")

genre_name_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["genre_name"]))
parent_id_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["parent_id"]))
parent_name_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["parent_name"]))
root_id_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["root_genre_id"]))
root_name_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["root_genre_name"]))

candidate_genre_ids = set(
    strategy_df.loc[strategy_df["stage_role"] == "Stage 1",
"genre_id"].astype(int).tolist()
)

rare_tail_genre_ids = set(
    strategy_df.loc[strategy_df["stage_role"] == "Stage 2",
"genre_id"].astype(int).tolist()
)

print("Candidate genre count:", len(candidate_genre_ids))
print("Rare-tail genre count:", len(rare_tail_genre_ids))
```

```
Candidate genre count: 150
Rare-tail genre count: 13
```

In [4]:

```
# ============================================================
# 4. IDENTIFY LABEL COLUMNS
# ============================================================

label_cols = [
    col for col in full_master_df.columns
    if col.startswith("genre_") and col.replace("genre_", "").isdigit()
]

label_col_ids = {int(col.replace("genre_", "")) for col in label_cols}

candidate_label_cols = [
    f"genre_{gid}" for gid in sorted(candidate_genre_ids) if f"genre_{gid}" in
full_master_df.columns
]

rare_tail_label_cols = [
    f"genre_{gid}" for gid in sorted(rare_tail_genre_ids) if f"genre_{gid}" in
full_master_df.columns
]

print("Total label columns in full master:", len(label_cols))
print("Candidate label columns found:", len(candidate_label_cols))
print("Rare-tail label columns found:", len(rare_tail_label_cols))
```

```
Total label columns in full master: 163
Candidate label columns found: 150
Rare-tail label columns found: 13
```

In [5]:

```
# ============================================================
# 5. HIERARCHY HELPER FUNCTIONS
# ============================================================

def normalize_int_or_none(x):
    if pd.isna(x):
        return None
    try:
        return int(x)
    except Exception:
        return None

def get_ancestor_chain(genre_id, parent_map):
    """
    Returns ancestor chain from immediate parent upward.
    Example: Tango -> Latin America -> International
    """
    chain = []
```

```
    visited = set()

    current = normalize_int_or_none(parent_map.get(genre_id, None))

    while current is not None and current not in visited:
        chain.append(current)
        visited.add(current)
        current = normalize_int_or_none(parent_map.get(current, None))

    return chain

def get_nearest_candidate_ancestor(genre_id, parent_map, candidate_set):
    chain = get_ancestor_chain(genre_id, parent_map)
    for anc in chain:
        if anc in candidate_set:
            return anc
    return None
```

In [6]:

```
# ============================================================
# 6. BUILD RARE-TAIL ROUTER TABLE
# ============================================================

router_rows = []

for _, row in rare_tail_summary_df.iterrows():
    gid = int(row["genre_id"])
    gname = row["genre_name"]

    ancestor_chain_ids = get_ancestor_chain(gid, parent_id_map)
    ancestor_chain_names = [genre_name_map.get(x) for x in ancestor_chain_ids]

    anchor_candidate_id = get_nearest_candidate_ancestor(gid, parent_id_map,
candidate_genre_ids)
    anchor_candidate_name = genre_name_map.get(anchor_candidate_id) if
anchor_candidate_id is not None else None

    root_candidate_id = normalize_int_or_none(root_id_map.get(gid, None))
    root_candidate_name = genre_name_map.get(root_candidate_id) if
root_candidate_id is not None else None

    if row["feasibility_status"] == "No train support":
        fallback_mode = "Inventory only"
    elif row["feasibility_status"] in ["Train only", "Partial split support",
"Extremely scarce", "Very scarce"]:
        fallback_mode = "Hierarchy-triggered fallback"
    else:
        fallback_mode = "Review"

    router_rows.append({
        "rare_tail_genre_id": gid,
        "rare_tail_genre_name": gname,
        "parent_id": normalize_int_or_none(row["parent_id"]),
        "parent_name": row["parent_name"],
```

```
        "root_genre_id": normalize_int_or_none(row["root_genre_id"]),
        "root_genre_name": row["root_genre_name"],
        "training_count": int(row["training_count"]),
        "validation_count": int(row["validation_count"]),
        "test_count": int(row["test_count"]),
        "total_count": int(row["total_count"]),
        "feasibility_status": row["feasibility_status"],
        "ancestor_chain_ids": " > ".join([str(x) for x in ancestor_chain_ids]),
        "ancestor_chain_names": " > ".join([str(x) for x in ancestor_chain_names]),
        "anchor_candidate_id": anchor_candidate_id,
        "anchor_candidate_name": anchor_candidate_name,
        "root_candidate_id": root_candidate_id,
        "root_candidate_name": root_candidate_name,
        "fallback_mode": fallback_mode
    })

rare_tail_router_df = pd.DataFrame(router_rows).sort_values(
    ["fallback_mode", "total_count", "training_count"],
    ascending=[True, False, False]
).reset_index(drop=True)

print("Rare-tail router table:")
display(rare_tail_router_df)
```

```
Rare-tail router table:
```

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre
0	176	Pacific	2	International	2	Interr
1	1060	Tango	46	Latin America	2	Interr
2	465	Musical Theater	20	Spoken	20	S
3	189	Talk Radio	65	Radio	20	S
4	1032	Turkish	102	Middle East	2	Interr
5	374	Banter	20	Spoken	20	S
6	173	N. Indian Traditional	86	Indian	2	Interr
7	493	Western Swing	651	Country & Western	9	C
8	377	Deep Funk	19	Funk	14	So
9	808	Salsa	46	Latin America	2	Interr
10	174	South Indian Traditional	86	Indian	2	Interr
11	175	Bollywood	86	Indian	2	Interr
12	178	Be-Bop	4	Jazz	4	

In [7]:

```
# ============================================================
# 7. COMPUTE TRAINING CO-OCCURRENCE STATISTICS
# ============================================================

train_df = full_master_df[full_master_df["split"] ==
"training"].copy().reset_index(drop=True)
```

```
cooccurrence_rows = []

for _, row in rare_tail_router_df.iterrows():
    rare_id = int(row["rare_tail_genre_id"])
    rare_col = f"genre_{rare_id}"

    anchor_id = row["anchor_candidate_id"]
    root_id = row["root_candidate_id"]

    rare_train_count = int(train_df[rare_col].sum()) if rare_col in
train_df.columns else 0

    # Anchor stats
    if anchor_id is not None and f"genre_{anchor_id}" in train_df.columns:
        anchor_col = f"genre_{anchor_id}"
        anchor_train_count = int(train_df[anchor_col].sum())
        cooccur_anchor_count = int(((train_df[rare_col] == 1) &
(train_df[anchor_col] == 1)).sum())
        p_rare_given_anchor = cooccur_anchor_count / anchor_train_count if
anchor_train_count > 0 else np.nan
        p_anchor_given_rare = cooccur_anchor_count / rare_train_count if
rare_train_count > 0 else np.nan
    else:
        anchor_train_count = 0
        cooccur_anchor_count = 0
        p_rare_given_anchor = np.nan
        p_anchor_given_rare = np.nan

    # Root stats
    if root_id is not None and f"genre_{root_id}" in train_df.columns:
        root_col = f"genre_{root_id}"
        root_train_count = int(train_df[root_col].sum())
        cooccur_root_count = int(((train_df[rare_col] == 1) & (train_df[root_col]
== 1)).sum())
        p_rare_given_root = cooccur_root_count / root_train_count if
root_train_count > 0 else np.nan
        p_root_given_rare = cooccur_root_count / rare_train_count if
rare_train_count > 0 else np.nan
    else:
        root_train_count = 0
        cooccur_root_count = 0
        p_rare_given_root = np.nan
        p_root_given_rare = np.nan

    cooccurrence_rows.append({
        "rare_tail_genre_id": rare_id,
        "rare_tail_genre_name": row["rare_tail_genre_name"],
        "training_count": rare_train_count,
        "anchor_candidate_id": anchor_id,
        "anchor_candidate_name": row["anchor_candidate_name"],
        "anchor_train_count": anchor_train_count,
        "cooccur_anchor_count": cooccur_anchor_count,
        "p_rare_given_anchor": p_rare_given_anchor,
        "p_anchor_given_rare": p_anchor_given_rare,
        "root_candidate_id": root_id,
        "root_candidate_name": row["root_candidate_name"],
        "root_train_count": root_train_count,
```

```
        "cooccur_root_count": cooccur_root_count,
        "p_rare_given_root": p_rare_given_root,
        "p_root_given_rare": p_root_given_rare
    })

cooccurrence_df = pd.DataFrame(cooccurrence_rows)

print("Rare-tail co-occurrence table:")
display(cooccurrence_df)
```

```
Rare-tail co-occurrence table:
```

	rare_tail_genre_id	rare_tail_genre_name	training_count	anchor_candidate_id	anchor_candic
0	176	Pacific	17	2	In
1	1060	Tango	5	46	Lat
2	465	Musical Theater	4	20	
3	189	Talk Radio	13	65	
4	1032	Turkish	10	102	N
5	374	Banter	5	20	
6	173	N. Indian Traditional	3	86	
7	493	Western Swing	1	651	Country
8	377	Deep Funk	1	19	
9	808	Salsa	1	46	Lat
10	174	South Indian Traditional	0	86	
11	175	Bollywood	0	86	
12	178	Be-Bop	0	4	

In [8]:

```
# ============================================================
# 8. MERGE ROUTER + CO-OCCURRENCE INTO FINAL ROUTING TABLE
# ============================================================

rare_tail_routing_table_df = rare_tail_router_df.merge(
    cooccurrence_df,
    on=["rare_tail_genre_id", "rare_tail_genre_name", "training_count",
"anchor_candidate_id",
        "anchor_candidate_name", "root_candidate_id", "root_candidate_name"],
    how="left"
)

print("Final rare-tail routing table:")
display(rare_tail_routing_table_df)
```

```
Final rare-tail routing table:
```

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre
0	176	Pacific	2	International	2	Intern
1	1060	Tango	46	Latin America	2	Intern
2	465	Musical Theater	20	Spoken	20	S
3	189	Talk Radio	65	Radio	20	S
4	1032	Turkish	102	Middle East	2	Intern
5	374	Banter	20	Spoken	20	S
6	173	N. Indian Traditional	86	Indian	2	Intern
7	493	Western Swing	651	Country & Western	9	C
8	377	Deep Funk	19	Funk	14	So
9	808	Salsa	46	Latin America	2	Intern
10	174	South Indian Traditional	86	Indian	2	Intern
11	175	Bollywood	86	Indian	2	Intern
12	178	Be-Bop	4	Jazz	4	

13 rows × 26 columns

In [9]:

```
# ============================================================
# 9. BUILD ANCHOR -> RARE-TAIL SUGGESTION TABLE
# ============================================================

anchor_to_tail_df = rare_tail_routing_table_df.copy()

anchor_to_tail_df["anchor_score"] =
anchor_to_tail_df["p_rare_given_anchor"].fillna(-1)
anchor_to_tail_df["root_score"] = anchor_to_tail_df["p_rare_given_root"].fillna(-1)

anchor_to_tail_df = anchor_to_tail_df.sort_values(
    ["anchor_candidate_name", "anchor_score", "training_count",
"rare_tail_genre_name"],
    ascending=[True, False, False, True]
).reset_index(drop=True)

anchor_to_tail_display_df = anchor_to_tail_df[
    [
        "anchor_candidate_id",
        "anchor_candidate_name",
        "rare_tail_genre_id",
        "rare_tail_genre_name",
        "training_count",
        "validation_count",
        "test_count",
        "p_rare_given_anchor",
        "p_anchor_given_rare",
        "fallback_mode"
    ]
].copy()

print("Anchor to rare-tail suggestion table:")
display(anchor_to_tail_display_df)
```

```
Anchor to rare-tail suggestion table:
```

	anchor_candidate_id	anchor_candidate_name	rare_tail_genre_id	rare_tail_genre_name	train
0	651	Country & Western	493	Western Swing	
1	19	Funk	377	Deep Funk	
2	86	Indian	173	N. Indian Traditional	
3	86	Indian	175	Bollywood	
4	86	Indian	174	South Indian Traditional	
5	2	International	176	Pacific	
6	4	Jazz	178	Be-Bop	
7	46	Latin America	1060	Tango	
8	46	Latin America	808	Salsa	
9	102	Middle East	1032	Turkish	
10	65	Radio	189	Talk Radio	
11	20	Spoken	374	Banter	
12	20	Spoken	465	Musical Theater	

In [10]:

```
# ============================================================
# 10. ROUTING COVERAGE CHECK USING GROUND-TRUTH TRACK LABELS
# ============================================================

full_master_indexed = full_master_df.set_index("track_id", drop=False)

coverage_rows = []
```

```
for _, row in rare_tail_routing_table_df.iterrows():
    rare_id = int(row["rare_tail_genre_id"])
    anchor_id = row["anchor_candidate_id"]
    root_id = row["root_candidate_id"]

    rare_pairs = rare_tail_pairs_df[rare_tail_pairs_df["rare_tail_genre_id"] ==
rare_id].copy()
    track_ids = rare_pairs["track_id"].astype(int).tolist()

    anchor_truth_hits = 0
    root_truth_hits = 0

    for tid in track_ids:
        track_row = full_master_indexed.loc[tid]

        if anchor_id is not None and f"genre_{anchor_id}" in
full_master_indexed.columns:
            anchor_truth_hits += int(track_row[f"genre_{anchor_id}"] == 1)

        if root_id is not None and f"genre_{root_id}" in
full_master_indexed.columns:
            root_truth_hits += int(track_row[f"genre_{root_id}"] == 1)

    total_pairs = len(track_ids)

    coverage_rows.append({
        "rare_tail_genre_id": rare_id,
        "rare_tail_genre_name": row["rare_tail_genre_name"],
        "total_pairs": total_pairs,
        "anchor_candidate_id": anchor_id,
        "anchor_candidate_name": row["anchor_candidate_name"],
        "anchor_truth_hits": anchor_truth_hits,
        "anchor_truth_coverage": anchor_truth_hits / total_pairs if total_pairs > 0
else np.nan,
        "root_candidate_id": root_id,
        "root_candidate_name": row["root_candidate_name"],
        "root_truth_hits": root_truth_hits,
        "root_truth_coverage": root_truth_hits / total_pairs if total_pairs > 0
else np.nan
    })

routing_coverage_df = pd.DataFrame(coverage_rows).sort_values(
    ["anchor_truth_coverage", "root_truth_coverage", "total_pairs"],
    ascending=[False, False, False]
).reset_index(drop=True)

print("Rare-tail routing coverage table:")
display(routing_coverage_df)
```

```
Rare-tail routing coverage table:
```

	rare_tail_genre_id	rare_tail_genre_name	total_pairs	anchor_candidate_id	anchor_candidate_
0	176	Pacific	23	2	Interna
1	1060	Tango	23	46	Latin A
2	465	Musical Theater	18	20	S
3	189	Talk Radio	15	65	
4	1032	Turkish	15	102	Midd
5	174	South Indian Traditional	15	86	
6	374	Banter	5	20	S
7	173	N. Indian Traditional	4	86	
8	493	Western Swing	4	651	Country & W
9	377	Deep Funk	1	19	
10	808	Salsa	1	46	Latin A
11	175	Bollywood	0	86	
12	178	Be-Bop	0	4	

In [11]:

```
# ============================================================
# 11. BUILD STAGE-2 EXECUTION STRATEGY TABLE
# ============================================================

stage2_strategy_rows = []

for _, row in rare_tail_routing_table_df.iterrows():
    gid = int(row["rare_tail_genre_id"])
    status = row["feasibility_status"]
    mode = row["fallback_mode"]

    if mode == "Inventory only":
        execution_rule = (
            "Do not predict directly. Keep in inventory and expose only as taxonomy
metadata or manual review label."
        )
    else:
        execution_rule = (
            "Only surface as a fallback suggestion when the anchor/root candidate
family is triggered by Stage 1."
        )

    if status in ["Very scarce", "Extremely scarce"]:
        recommendation = "Rare-tail hint only"
    elif status == "Partial split support":
        recommendation = "Hierarchy fallback only"
    elif status == "Train only":
```

```
        recommendation = "Training-only tail; no reliable evaluation support"
    elif status == "No train support":
        recommendation = "Inventory only"
    else:
        recommendation = "Review manually"

    stage2_strategy_rows.append({
        "rare_tail_genre_id": gid,
        "rare_tail_genre_name": row["rare_tail_genre_name"],
        "anchor_candidate_name": row["anchor_candidate_name"],
        "root_candidate_name": row["root_candidate_name"],
        "feasibility_status": status,
        "fallback_mode": mode,
        "execution_rule": execution_rule,
        "recommendation": recommendation
    })

stage2_strategy_df = pd.DataFrame(stage2_strategy_rows)

print("Stage-2 execution strategy:")
display(stage2_strategy_df)
```

```
Stage-2 execution strategy:
```

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	feas
0	176	Pacific	International	International	
1	1060	Tango	Latin America	International	Ext
2	465	Musical Theater	Spoken	Spoken	Ext
3	189	Talk Radio	Radio	Spoken	
4	1032	Turkish	Middle East	International	
5	374	Banter	Spoken	Spoken	
6	173	N. Indian Traditional	Indian	International	
7	493	Western Swing	Country & Western	Country	Ext
8	377	Deep Funk	Funk	Soul-RnB	
9	808	Salsa	Latin America	International	
10	174	South Indian Traditional	Indian	International	No
11	175	Bollywood	Indian	International	No

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	feas
12	178	Be-Bop	Jazz	Jazz	Nc

In [12]:

```
# ============================================================
# 12. BUILD FULL-161 READINESS SUMMARY
# ============================================================

num_inventory_only = int((stage2_strategy_df["fallback_mode"] == "Inventory
only").sum())
num_hierarchy_fallback = int((stage2_strategy_df["fallback_mode"] == "Hierarchy-
triggered fallback").sum())
num_with_anchor = int(stage2_strategy_df["anchor_candidate_name"].notna().sum())

readiness_rows = [
    {
        "Component": "Stage 1 candidate-150 benchmark model",
        "Status": "Ready",
        "Details": "Expanded hybrid benchmark system is already frozen."
    },
    {
        "Component": "Stage 2 rare-tail router",
        "Status": "Ready",
        "Details": f"Built routing table for {len(stage2_strategy_df)} rare-tail
labels."
    },
    {
        "Component": "Rare-tail labels with candidate anchors",
        "Status": "Ready",
        "Details": f"{num_with_anchor} rare-tail labels map to a nearest candidate
ancestor."
    },
    {
        "Component": "Hierarchy-triggered fallback labels",
        "Status": "Ready",
        "Details": f"{num_hierarchy_fallback} labels can be surfaced through
hierarchy-based fallback."
    },
    {
        "Component": "Inventory-only rare-tail labels",
        "Status": "Ready",
        "Details": f"{num_inventory_only} labels should remain inventory-only until
stronger support exists."
    },
    {
        "Component": "Full 161-label inference pipeline",
        "Status": "Next step",
```

```
        "Details": "Combine Stage 1 benchmark predictions with Stage 2 fallback
routing in the next notebook."
    }
]

full161_readiness_df = pd.DataFrame(readiness_rows)

print("Full-161 readiness summary:")
display(full161_readiness_df)
```

```
Full-161 readiness summary:
```

	Component	Status	Details
0	Stage 1 candidate-150 benchmark model	Ready	Expanded hybrid benchmark system is already fr...
1	Stage 2 rare-tail router	Ready	Built routing table for 13 rare-tail labels.
2	Rare-tail labels with candidate anchors	Ready	13 rare-tail labels map to a nearest candidate...
3	Hierarchy-triggered fallback labels	Ready	10 labels can be surfaced through hierarchy-ba...
4	Inventory-only rare-tail labels	Ready	3 labels should remain inventory-only until st...
5	Full 161-label inference pipeline	Next step	Combine Stage 1 benchmark predictions with Sta...

In [13]:

```
# ============================================================
# 13. SAVE OUTPUTS
# ============================================================

os.makedirs("../data/processed", exist_ok=True)

rare_tail_routing_table_df.to_csv(
    "../data/processed/full161_rare_tail_routing_table.csv",
    index=False
)

anchor_to_tail_display_df.to_csv(
    "../data/processed/full161_anchor_to_rare_tail_table.csv",
    index=False
)

routing_coverage_df.to_csv(
    "../data/processed/full161_rare_tail_routing_coverage.csv",
    index=False
)

stage2_strategy_df.to_csv(
    "../data/processed/full161_stage2_execution_strategy.csv",
    index=False
```

```
)

full161_readiness_df.to_csv(
    "../data/processed/full161_readiness_summary.csv",
    index=False
)

print("Saved full-161 rare-tail fallback router outputs.")
```

```
Saved full-161 rare-tail fallback router outputs.
```

In [14]:

```
# ============================================================
# 14. INTERPRETATION NOTES
# ============================================================

print("1. The rare-tail stage should be handled through hierarchy-triggered
fallback, not direct conventional training.")
print("2. This notebook maps each rare-tail label to a candidate ancestor and
computes fallback statistics.")
print("3. The outputs now define how Stage 2 should behave in a full 161-label
pipeline.")
print("4. The next notebook should combine the frozen candidate-150 benchmark model
with this rare-tail router.")
print("5. After that, the project will have a working full-genre inference pipeline
design.")
```

```
1. The rare-tail stage should be handled through hierarchy-triggered fallback, not
direct conventional training.
2. This notebook maps each rare-tail label to a candidate ancestor and computes
fallback statistics.
3. The outputs now define how Stage 2 should behave in a full 161-label pipeline.
4. The next notebook should combine the frozen candidate-150 benchmark model with
this rare-tail router.
5. After that, the project will have a working full-genre inference pipeline design.
```